DR. RVR NRI INSTITUTE OF TECHNOLOGY

PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING CONSOLIDATED REPORT

10 Selected Difficult-Level Case Studies

Student Name	_____
Roll Number	_____
Branch / Section	_____
Course	Python Programming
Academic Year	2026–2027

Prepared according to the college circular and the supplied Python ABL question bank.

Before submission, fill in student details, run every program, and capture genuine screenshots from your Python environment if required by your faculty.

CONTENTS

Each case study contains all ten required sections: problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output sample, and conclusion.

Case Study 12 — Attendance Recovery Computation

Case Study 13 — ATM Transaction Sequence Simulation

Case Study 17 — Student Performance Trend Analysis

Case Study 27 — Advanced Numeric Pattern Generator

Case Study 29 — Password Policy Validation Engine

Case Study 35 — Text Compression Using Character Runs

Case Study 42 — Student Marks Matrix Analysis

Case Study 48 — Tuple-Based Employee Shift Conflict Detection

Case Study 49 — Tuple-Based Product Catalogue Audit

Case Study 57 — Smart Canteen Order and Stock Management

Selected case-study numbers: 12, 13, 17, 27, 29, 35, 42, 48, 49, and 57.

CASE STUDY 12: Attendance Recovery Computation

1. Problem Statement

Calculate a student's current attendance percentage and determine the minimum number of consecutive classes the student must attend to reach a required attendance threshold.

2. Objectives

• Calculate attendance percentage from attended and conducted classes.

• Use a loop to find the minimum additional classes needed.

• Validate inputs and handle cases where no classes have yet been conducted.

3. Assumptions

• Class counts are non-negative integers; attended classes cannot exceed total classes.

• The target percentage must be between 0 and 100 inclusive.

• Each future class attended increases both attended and total class counts by one.

4. Algorithm

1. Read attended classes, total classes, and target percentage.

2. Validate all values.

3. If total is zero, report that the percentage is undefined.

4. Otherwise calculate the current percentage.

5. Increase a counter until the projected attendance reaches the target.

6. Display the current percentage, extra classes needed, and projected percentage.

5. Flowchart

```
[START] → [Input attended, total, target] → [Validate input]
                          ↓ valid
                 [total = 0?] — yes → [Report undefined] → [END]
                          no
                 [Calculate percentage]
                          ↓
                 [While projected % < target: extra += 1]
                          ↓
                 [Display result] → [END]
```

6. Pseudocode

```
START
INPUT attended, total, target
IF attended < 0 OR total < 0 OR attended > total OR target < 0 OR target > 100
    PRINT "Invalid input"
ELSE IF total = 0
    PRINT "Attendance percentage is undefined"
ELSE
    current = attended / total * 100
    extra = 0
    WHILE (attended + extra) / (total + extra) * 100 < target
        extra = extra + 1
    END WHILE
    PRINT current, extra, projected percentage
END IF
STOP
```

7. Python Source Code

```
attended = int(input("Classes attended: "))
total = int(input("Total classes conducted: "))
target = float(input("Required attendance percentage: "))

if attended < 0 or total < 0 or attended > total or not 0 <= target <= 100:
    print("Invalid input. Check the class counts and target percentage.")
elif total == 0:
    print("Attendance percentage is undefined because no classes were conducted.")
else:
    current = attended / total * 100
    extra = 0
    while (attended + extra) / (total + extra) * 100 < target:
        extra += 1
    print("Current attendance: {:.2f}%".format(current))
```

```
print("Minimum consecutive classes to attend:", extra)
print("Projected attendance: {:.2f}%".format(
    (attended + extra) / (total + extra) * 100
))
```

8. Test Cases

Input / Condition	Expected Result
24, 40, 75	60.00%; 24 extra classes; projected 62.50%
24, 40, 75	Current attendance 60.00%; minimum 24 extra classes; projected attendance 75.00%
0, 0, 75	Attendance percentage is undefined

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.

Sample Python console output

```
Classes attended: 24
Total classes conducted: 40
Required attendance percentage: 75
Current attendance: 60.00%
Minimum consecutive classes to attend: 24
Projected attendance: 75.00%
```

10. Conclusion

The program calculates current attendance and uses iteration to find the smallest number of future classes needed to reach the required threshold. Input validation prevents invalid class counts.

CASE STUDY 13: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate deposits and withdrawals, maintain a balance, reject invalid transactions, enforce a withdrawal limit, and produce a transaction summary.

2. Objectives

• Update the balance correctly for deposits and withdrawals.

• Reject invalid amounts, unknown transaction types, over-limit withdrawals, and insufficient-fund withdrawals.

• Summarize successful and rejected transactions.

3. Assumptions

• The opening balance is non-negative.

• Transaction type is D for deposit or W for withdrawal.

• The maximum withdrawal per transaction is ₹10,000.

• The program is a simulation and does not access a real bank account.

4. Algorithm

1. Read opening balance and number of transactions.

2. For each transaction, read type and amount.

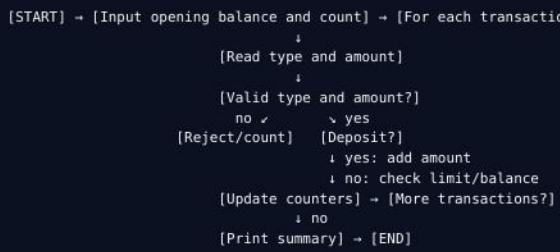
3. Reject an invalid type or non-positive amount.

4. For deposits, add the amount to the balance.

5. For withdrawals, check the limit and available balance before subtracting.

6. Print the final balance and transaction counts.

5. Flowchart



6. Pseudocode

```
START
INPUT balance, n
SET deposits = 0, withdrawals = 0, rejected = 0
REPEAT n times
    INPUT type, amount
    IF amount <= 0 OR type NOT IN {D, W}
        rejected = rejected + 1
    ELSE IF type = D
        balance = balance + amount
        deposits = deposits + 1
    ELSE IF amount > 10000 OR amount > balance
        rejected = rejected + 1
    ELSE
        balance = balance - amount
        withdrawals = withdrawals + 1
    END IF
END REPEAT
PRINT balance, deposits, withdrawals, rejected
STOP
```

7. Python Source Code

```
balance = float(input("Opening balance (Rs): "))
```

```
n = int(input("Number of transactions: "))

if balance < 0 or n < 0:
    print("Invalid opening balance or transaction count.")
else:
    deposits = 0
    withdrawals = 0
    rejected = 0

    for i in range(n):
        kind = input("Transaction type (D/W): ").strip().upper()
        amount = float(input("Amount (Rs): "))

        if amount <= 0 or kind not in ("D", "W"):
            print("Transaction rejected: invalid type or amount.")
            rejected += 1
        elif kind == "D":
            balance += amount
            deposits += 1
            print("Deposit successful.")
        elif amount > 10000:
            print("Transaction rejected: withdrawal limit is Rs 10000.")
            rejected += 1
        elif amount > balance:
            print("Transaction rejected: insufficient balance.")
            rejected += 1
        else:
            balance -= amount
            withdrawals += 1
            print("Withdrawal successful.")

    print("\n--- Transaction Summary ---")
    print("Final balance: Rs {:.2f}".format(balance))
    print("Successful deposits:", deposits)
    print("Successful withdrawals:", withdrawals)
    print("Rejected transactions:", rejected)
```

8. Test Cases

Input / Condition	Expected Result
Opening 5000; D 1000; W 2000	Final balance Rs 4000.00; 1 deposit; 1 withdrawal; 0 rejected
Opening 500; W 700	Rejected for insufficient balance; balance remains Rs 500.00
Opening 15000; W 12000	Rejected because withdrawal exceeds Rs 10000 limit

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.

Sample Python console output

```
Opening balance (Rs): 5000
Number of transactions: 3
Transaction type (D/W): D
Amount (Rs): 1000
Deposit successful.
Transaction type (D/W): W
Amount (Rs): 2000
Withdrawal successful.
Transaction type (D/W): W
Amount (Rs): 7000
Transaction rejected: insufficient balance.

--- Transaction Summary ---
Final balance: Rs 4000.00
Successful deposits: 1
Successful withdrawals: 1
Rejected transactions: 1
```

10. Conclusion

The simulation applies conditional rules to a sequence of transactions and maintains a consistent balance. It demonstrates loops, validation, counters, and formatted output.

CASE STUDY 17: Student Performance Trend Analysis

1. Problem Statement

Compare a student's scores across several examinations, calculate changes between exams, identify the strongest and weakest subjects, and summarize performance trends.

2. Objectives

• Store marks in lists and calculate totals and averages.

• Calculate score changes between consecutive exams.

• Identify highest- and lowest-scoring subjects.

3. Assumptions

• The sample program uses the same subjects and maximum mark of 100 for each exam.

• Marks are numeric and between 0 and 100.

• The demonstration uses three examinations and four subjects.

4. Algorithm

1. Define subject names and marks for three exams.

2. Validate that all marks lie between 0 and 100.

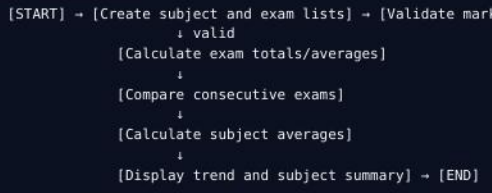
3. Calculate each exam's total and average.

4. Compare each later exam with the previous exam.

5. Calculate each subject's average across exams.

6. Display overall trend and strongest/weakest subject.

5. Flowchart



6. Pseudocode

```
START
SET subjects and marks for each exam
IF any mark is outside 0..100
    PRINT invalid marks
ELSE
    FOR each exam
        calculate total and average
    END FOR
    FOR each pair of consecutive exams
        calculate average change
    END FOR
    FOR each subject
        calculate average across exams
    END FOR
    PRINT summaries and best/worst subject
END IF
STOP
```

7. Python Source Code

```
subjects = ["Python", "Mathematics", "English", "Physics"]
exam1 = [72, 65, 80, 68]
exam2 = [78, 70, 82, 74]
exam3 = [85, 76, 79, 81]
exams = [exam1, exam2, exam3]

valid = True
for exam in exams:
    for mark in exam:
        if mark < 0 or mark > 100:
            valid = False
```

```
if not valid:
    print("Invalid marks: each mark must be from 0 to 100.")
else:
    averages = []
    for i, exam in enumerate(exams):
        total = sum(exam)
        average = total / len(exam)
        averages.append(average)
        print("Exam {} total: {}, average: {:.2f}".format(
            i + 1, total, average
        ))

    print("Average change, Exam 1 to 2: {:+.2f}".format(
        averages[1] - averages[0]
    ))
    print("Average change, Exam 2 to 3: {:+.2f}".format(
        averages[2] - averages[1]
    ))

    subject_averages = []
    for j, subject in enumerate(subjects):
        avg = (exam1[j] + exam2[j] + exam3[j]) / 3
        subject_averages.append(avg)
        print("{} average: {:.2f}".format(subject, avg))

    best = subject_averages.index(max(subject_averages))
    worst = subject_averages.index(min(subject_averages))
    print("Strongest subject:", subjects[best])
    print("Subject needing most improvement:", subjects[worst])
```

8. Test Cases

Input / Condition	Expected Result
Given marks in code	Exam averages: 71.25, 76.00, 80.25; strongest subject English; weakest Mathematics
Change a mark to 105	Invalid marks message

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.

Sample Python console output

```
Exam 1 total: 285, average: 71.25
Exam 2 total: 304, average: 76.00
Exam 3 total: 321, average: 80.25
Average change, Exam 1 to 2: +4.75
Average change, Exam 2 to 3: +4.25
Python average: 78.33
Mathematics average: 70.33
English average: 80.33
Physics average: 74.33
Strongest subject: English
Subject needing most improvement: Mathematics
```

10. Conclusion

The program uses nested lists and loops to compare examination performance over time. It highlights changes in average marks and helps identify subjects that may need additional attention.

CASE STUDY 27: Advanced Numeric Pattern Generator

1. Problem Statement

Generate a symmetric hollow diamond pattern based on a user-provided odd width, and reject dimensions that cannot form the requested pattern.

2. Objectives

• Use nested loops to generate a symmetric pattern.

• Use conditional statements to print boundary characters and spaces.

• Validate the input dimension.

3. Assumptions

• The width must be an odd integer between 3 and 19 inclusive.

• The pattern uses asterisks on the outline and spaces inside.

• The upper and lower halves are symmetric.

4. Algorithm

1. Read the requested width.

2. Check that it is odd and within the permitted range.

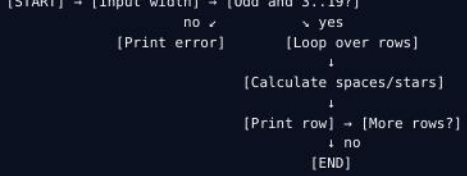
3. Set the middle row index to width divided by two.

4. For each row, calculate leading spaces and the distance from the middle.

5. Print one star for the top/bottom tip or two outline stars for other rows, with interior spaces.

6. Print rows from top to bottom to create a hollow diamond.

5. Flowchart



6. Pseudocode

```
START
INPUT width
IF width < 3 OR width > 19 OR width is even
    PRINT invalid width
ELSE
    middle = width // 2
    FOR row FROM 0 TO width - 1
        distance = absolute value(row - middle)
        leading = distance
        inner = width - 2 * distance - 2
        PRINT leading spaces
        IF inner < 0
            PRINT "*"
        ELSE
            PRINT "*", inner spaces, "*"
        END IF
    END FOR
END IF
STOP
```

7. Python Source Code

```
width = int(input("Enter an odd width (3 to 19): "))

if width < 3 or width > 19 or width % 2 == 0:
    print("Invalid width. Enter an odd number from 3 to 19.")
else:
    middle = width // 2
    for row in range(width):
```

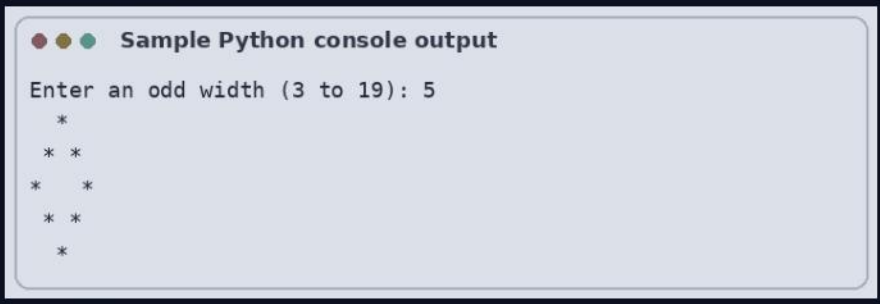
```
    distance = abs(row - middle)
    leading = distance
    inner = width - 2 * distance - 2
    print(" " * leading, end="")
    if inner < 0:
        print("*")
    else:
        print("*" + " " * inner + "*")
```

8. Test Cases

Input / Condition	Expected Result
5	*\n * *\n* *\n * *\n *
4	Invalid width. Enter an odd number from 3 to 19.
21	Invalid width. Enter an odd number from 3 to 19.

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.



10. Conclusion

The program demonstrates nested loops, arithmetic, string repetition, and input validation to construct a symmetric pattern. The width restriction keeps the output manageable and correctly aligned.

CASE STUDY 29: Password Policy Validation Engine

1. Problem Statement

Evaluate a sample password against minimum length, uppercase, lowercase, digit, special-character, repeated-character, and prohibited-substring rules, then report every failed condition.

2. Objectives

• Inspect a string using loops and character tests.

• Apply several independent password rules.

• Report all failed rules instead of stopping at the first failure.

3. Assumptions

• A valid password must contain at least 8 characters.

• It must contain uppercase, lowercase, digit, and special characters.

• It must not contain three identical consecutive characters.

• The prohibited substring is the case-insensitive word 'password'.

4. Algorithm

1. Read a password string.

2. Check its length.

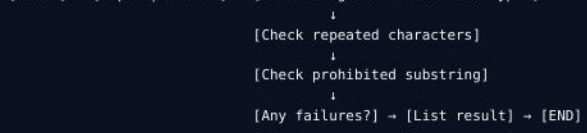
3. Loop through characters to detect uppercase, lowercase, digits, and special characters.

4. Check for three identical consecutive characters.

5. Convert to lowercase and check for the prohibited substring.

6. Display each failed rule or report that all rules passed.

5. Flowchart



6. Pseudocode

```
START
INPUT password
SET failures = empty list
IF length(password) < 8: add length rule to failures
SCAN characters for uppercase, lowercase, digit, special character
ADD any missing categories to failures
CHECK every group of 3 consecutive characters
IF any group is identical: add repetition rule
IF "password" occurs in lowercase password: add prohibited-word rule
IF failures is empty PRINT "All rules passed"
ELSE PRINT each failure
STOP
```

7. Python Source Code

```
password = input("Enter a sample password: ")
failures = []

if len(password) < 8:
    failures.append("Use at least 8 characters.")

has_upper = False
has_lower = False
has_digit = False
has_special = False

for ch in password:
    if ch.isupper():
        has_upper = True
    elif ch.islower():
        has_lower = True
    elif ch.isdigit():
```

```
        has_digit = True
    else:
        has_special = True

if not has_upper:
    failures.append("Include an uppercase letter.")
if not has_lower:
    failures.append("Include a lowercase letter.")
if not has_digit:
    failures.append("Include a digit.")
if not has_special:
    failures.append("Include a special character.")

repeated = False
for i in range(len(password) - 2):
    if password[i] == password[i + 1] == password[i + 2]:
        repeated = True
if repeated:
    failures.append("Do not use three identical consecutive characters.")

if "password" in password.lower():
    failures.append("Do not include the word 'password'.")

if failures:
    print("Password does not meet the policy:")
    for item in failures:
        print("-", item)
else:
    print("All password rules passed.")
```

8. Test Cases

Input / Condition	Expected Result
Abc@1234	All password rules passed.
abc	Fails length, uppercase, digit, special-character rules
Password111!	Fails prohibited substring and repeated-character rule

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.

Sample Python console output

```
Enter a sample password: abc
Password does not meet the policy:
- Use at least 8 characters.
- Include an uppercase letter.
- Include a digit.
- Include a special character.
```

10. Conclusion

The validator combines string operations, loops, Boolean flags, and conditions. It gives clear feedback about each failed rule. It is an educational example, not a complete production security checker.

CASE STUDY 35: Text Compression Using Character Runs

1. Problem Statement

Encode a string by grouping consecutive repeated characters and compare the encoded length with the original length. If encoding is not shorter, keep the original string.

2. Objectives

• Identify consecutive runs of the same character.

• Build a run-length encoded string using a loop.

• Compare lengths and choose the shorter representation.

3. Assumptions

• The input may contain letters, digits, spaces, and punctuation.

• Each run is represented as character followed by its count, such as A4.

• If encoded output is not shorter, the original text is selected.

4. Algorithm

1. Read the input string.

2. If it is empty, display an empty result.

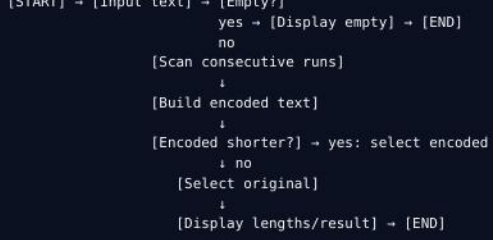
3. Otherwise scan characters from left to right and count each consecutive run.

4. Append each run as character plus count.

5. Compare original and encoded lengths.

6. Display the encoded candidate and the selected representation.

5. Flowchart



6. Pseudocode

```
START
INPUT text
IF text is empty
    PRINT empty result
ELSE
    SET encoded = "", count = 1
    FOR i FROM 1 TO length(text)-1
        IF text[i] = text[i-1]
            count = count + 1
        ELSE
            encoded = encoded + text[i-1] + count
            count = 1
        END IF
    END FOR
    encoded = encoded + last character + count
    IF length(encoded) < length(text): selected = encoded
    ELSE: selected = text
    PRINT encoded and selected
END IF
STOP
```

7. Python Source Code

```
text = input("Enter text to compress: ")
```

```
if text == "":
    print("Input is empty.")
else:
    encoded = ""
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            encoded += text[i - 1] + str(count)
            count = 1

    encoded += text[-1] + str(count)

    print("Original text:", text)
    print("Encoded candidate:", encoded)
    print("Original length:", len(text))
    print("Encoded length:", len(encoded))

    if len(encoded) < len(text):
        print("Selected representation:", encoded)
    else:
        print("Encoding is not beneficial; keep original text.")
        print("Selected representation:", text)
```

8. Test Cases

Input / Condition	Expected Result
AAAAABBBCC	A5B3C2; encoded is shorter
ABCD	A1B1C1D1; original is shorter
	Input is empty

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.



10. Conclusion

The program applies a basic run-length encoding method and checks whether it saves space. It illustrates strings, indexing, loops, and conditional selection.

CASE STUDY 42: Student Marks Matrix Analysis

1. Problem Statement

Analyse a matrix of student marks across subjects, calculate student totals and averages, calculate subject averages, identify the top performer, and flag subjects with low average marks.

2. Objectives

• Represent marks using a nested list.

• Calculate row-wise and column-wise statistics.

• Identify the top student and subjects below a defined average threshold.

3. Assumptions

• Each row represents one student and each column represents one subject.

• Marks are integers between 0 and 100.

• A subject is flagged when its average is below 50.

• The sample uses four students and three subjects.

4. Algorithm

1. Create student names, subject names, and the marks matrix.

2. Validate every mark.

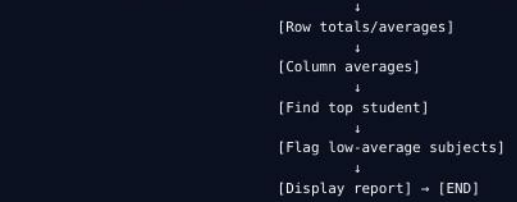
3. For each student, calculate total and average.

4. For each subject, sum the marks from all students and calculate the average.

5. Find the student with the highest total.

6. Flag subjects with averages below 50 and display all results.

5. Flowchart



6. Pseudocode

```
START
SET students, subjects, and marks matrix
VALIDATE each mark is between 0 and 100
FOR each student row
    total = sum(row)
    average = total / number of subjects
    PRINT total and average
END FOR
FOR each subject column
    calculate column average
    IF average < 50: flag subject
END FOR
FIND student with maximum total
PRINT top student and flagged subjects
STOP
```

7. Python Source Code

```
students = ["Asha", "Bharat", "Charan", "Divya"]
subjects = ["Python", "Maths", "English"]
marks = [
    [82, 76, 88],
    [65, 58, 72],
    [91, 85, 90],
    [42, 48, 55]
]
```

```
valid = True
for row in marks:
    for mark in row:
        if mark < 0 or mark > 100:
            valid = False

if not valid:
    print("Invalid marks: each mark must be between 0 and 100.")
else:
    totals = []
    for i in range(len(students)):
        total = sum(marks[i])
        average = total / len(subjects)
        totals.append(total)
        print("{}: total = {}, average = {:.2f}".format(
            students[i], total, average
        ))

    subject_averages = []
    for j in range(len(subjects)):
        total = 0
        for i in range(len(students)):
            total += marks[i][j]
        average = total / len(students)
        subject_averages.append(average)
        print("{} average: {:.2f}".format(subjects[j], average))

    top_index = totals.index(max(totals))
    print("Top performer:", students[top_index])
    print("Subjects below 50 average:")
    found = False
    for j in range(len(subjects)):
        if subject_averages[j] < 50:
            print("-", subjects[j])
            found = True
    if not found:
        print("None")
```

8. Test Cases

Input / Condition	Expected Result
Matrix in code	Top performer: Charan; no subject average below 50
Any mark set to 120	Invalid marks message

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.

Sample Python console output

```
Asha: total = 246, average = 82.00
Bharat: total = 195, average = 65.00
Charan: total = 266, average = 88.67
Divya: total = 145, average = 48.33
Python average: 70.00
Maths average: 66.75
English average: 76.25
Top performer: Charan
Subjects below 50 average:
None
```

10. Conclusion

The program uses nested lists and loops to analyse marks by student and by subject. It identifies the top total and highlights subjects that fall below the chosen average threshold.

CASE STUDY 48: Tuple-Based Employee Shift Conflict Detection

1. Problem Statement

Represent employee shifts as tuples, compare start and end times, detect overlapping assignments, and report employees with conflicting schedules.

2. Objectives

• Store shift records as tuples.

• Compare time intervals using conditional statements and nested loops.

• Report overlapping shifts assigned to the same employee.

3. Assumptions

• Times are represented as integer hours from 0 to 23.

• A shift ends after it starts and does not cross midnight.

• Two shifts overlap when each starts before the other ends.

• Shift records use (employee, start_hour, end_hour).

4. Algorithm

1. Create a list of shift tuples.

2. Check that each start time is earlier than its end time and both are valid hours.

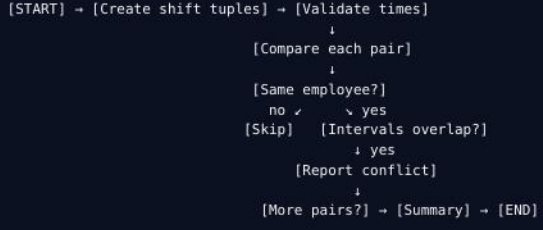
3. Compare every pair of shifts.

4. Ignore pairs belonging to different employees and a shift compared with itself.

5. If start1 < end2 and start2 < end1, report a conflict.

6. If no conflicts are found, report that all shifts are valid.

5. Flowchart



6. Pseudocode

```
START
SET shifts as tuples (employee, start, end)
VALIDATE 0 <= start < end <= 24
SET conflict_found = false
FOR i from 0 to number_of_shifts - 1
    FOR j from i + 1 to number_of_shifts - 1
        IF same employee AND start_i < end_j AND start_j < end_i
            PRINT conflicting shifts
            conflict_found = true
        END IF
    END FOR
END FOR
IF not conflict_found PRINT no conflicts
STOP
```

7. Python Source Code

```
shifts = [
    ("Ravi", 9, 13),
    ("Ravi", 12, 16),
    ("Meena", 9, 12),
    ("Ravi", 16, 18),
    ("Meena", 11, 14)
]
```

```
valid = True
for employee, start, end in shifts:
    if start < 0 or end > 24 or start >= end:
        valid = False

if not valid:
    print("Invalid shift time. Use 0 <= start < end <= 24.")
else:
    conflict_found = False
    for i in range(len(shifts)):
        for j in range(i + 1, len(shifts)):
            emp1, start1, end1 = shifts[i]
            emp2, start2, end2 = shifts[j]
            if emp1 == emp2 and start1 < end2 and start2 < end1:
                print("{} has overlapping shifts: {}-{} and {}-{}".format(
                    emp1, start1, end1, start2, end2
                ))
                conflict_found = True

    if not conflict_found:
        print("No shift conflicts found.")
```

8. Test Cases

Input / Condition	Expected Result
Ravi 9-13 and 12-16	Conflict detected because the intervals overlap
Ravi 9-13 and 13-16	No conflict; one shift ends when the next begins
Start 15, end 10	Invalid shift time

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.



10. Conclusion

The program demonstrates tuples, nested loops, and interval comparisons. It detects conflicting assignments for the same employee while allowing shifts that meet exactly at a boundary.

CASE STUDY 49: Tuple-Based Product Catalogue Audit

1. Problem Statement

Store product identifiers, prices, and stock quantities in tuples; detect invalid records, calculate inventory values, and identify products that need replenishment.

2. Objectives

• Represent catalogue entries as tuples.

• Validate prices and quantities.

• Calculate stock value and identify products below a minimum stock level.

3. Assumptions

• Each tuple is (product_id, price, quantity).

• Price and quantity cannot be negative.

• The minimum stock level is 5 units.

• The sample catalogue is hypothetical.

4. Algorithm

1. Create a list of product tuples.

2. Check each record for a non-empty ID and non-negative price and quantity.

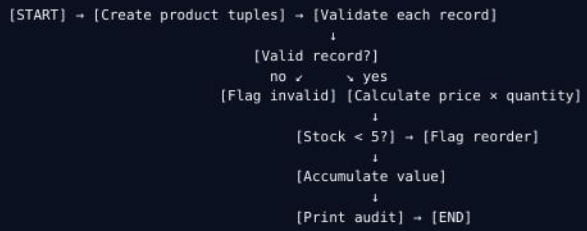
3. For valid records, calculate price multiplied by quantity.

4. Add each value to the total inventory value.

5. Flag products whose quantity is below five.

6. Print product details, invalid records, replenishment list, and total value.

5. Flowchart



6. Pseudocode

```
START
SET catalogue of (id, price, quantity) tuples
SET total_value = 0
FOR each product tuple
    IF id empty OR price < 0 OR quantity < 0
        PRINT invalid record
    ELSE
        value = price * quantity
        total_value = total_value + value
        PRINT product value
        IF quantity < 5: PRINT replenishment required
    END IF
END FOR
PRINT total_value
STOP
```

7. Python Source Code

```
catalogue = [
    ("P101", 120.0, 10),
    ("P102", 75.5, 3),
    ("P103", 250.0, 0),
    ("P104", 40.0, 8),
    ("P105", -20.0, 4)
]
minimum_stock = 5
total_value = 0
```

```
reorder_list = []

for product_id, price, quantity in catalogue:
    if product_id == "" or price < 0 or quantity < 0:
        print("Invalid record:", (product_id, price, quantity))
    else:
        value = price * quantity
        total_value += value
        print("{}: stock value = Rs {:.2f}".format(product_id, value))
        if quantity < minimum_stock:
            reorder_list.append(product_id)

print("Products needing replenishment:", reorder_list)
print("Total valid inventory value: Rs {:.2f}".format(total_value))
```

8. Test Cases

Input / Condition	Expected Result
P101 at Rs120, quantity 10	Stock value Rs 1200.00; not on reorder list
P102 at Rs75.50, quantity 3	Stock value Rs 226.50; replenishment required
Negative price	Invalid record flagged

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.

Sample Python console output

```
P101: stock value = Rs 1200.00
P102: stock value = Rs 226.50
P103: stock value = Rs 0.00
P104: stock value = Rs 320.00
Invalid record: ('P105', -20.0, 4)
Products needing replenishment: ['P102', 'P103']
Total valid inventory value: Rs 1746.50
```

10. Conclusion

The catalogue audit uses tuples to keep product fields together and loops to validate and calculate stock values. It identifies low-stock products and excludes invalid records from the total.

CASE STUDY 57: Smart Canteen Order and Stock Management

1. Problem Statement

Process successive canteen orders, calculate bills, update inventory, reject orders that exceed available stock, and summarize sales and remaining stock.

2. Objectives

• Maintain menu prices and stock quantities in lists.

• Calculate order totals and update stock after valid orders.

• Reject unavailable or invalid quantities and summarize sales.

3. Assumptions

• The menu and prices are fixed for this simulation.

• Order quantities must be positive integers.

• An order is accepted only if enough stock is available.

• The sample handles one order per menu item and uses rupees.

4. Algorithm

1. Define menu items, prices, and starting stock.

2. Read the number of order lines.

3. For each order, read item name and quantity.

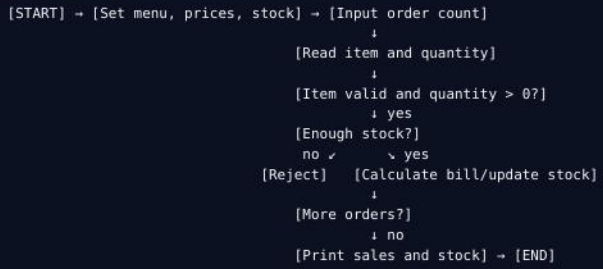
4. Find the item and validate the quantity and stock.

5. If valid, calculate the bill, reduce stock, and add to sales total.

6. Otherwise reject the order without changing stock.

7. Print daily sales and remaining stock.

5. Flowchart



6. Pseudocode

```
START
SET items, prices, stock
INPUT order_count
SET sales = 0
REPEAT order_count times
    INPUT item, quantity
    IF item not in menu OR quantity <= 0
        PRINT rejected order
    ELSE IF quantity > stock[item]
        PRINT insufficient stock
    ELSE
        bill = price[item] * quantity
        stock[item] = stock[item] - quantity
        sales = sales + bill
        PRINT bill
    END IF
END REPEAT
PRINT sales and remaining stock
STOP
```

7. Python Source Code

```
items = ["Idli", "Dosa", "Tea"]
prices = [30, 50, 15]
stock = [20, 10, 30]
sales = 0

order_count = int(input("Number of order lines: "))
if order_count < 0:
    print("Invalid order count.")
else:
    for _ in range(order_count):
        item = input("Item name (Idli/Dosa/Tea): ").strip().title()
        quantity = int(input("Quantity: "))

        if item not in items or quantity <= 0:
            print("Order rejected: invalid item or quantity.")
        else:
            index = items.index(item)
            if quantity > stock[index]:
                print("Order rejected: insufficient stock.")
            else:
                bill = prices[index] * quantity
                stock[index] -= quantity
                sales += bill
                print("Order accepted. Bill: Rs", bill)

    print("\n--- Daily Summary ---")
    print("Total sales: Rs", sales)
    print("Remaining stock:")
    for i in range(len(items)):
        print(items[i] + ":", stock[i])
```

8. Test Cases

Input / Condition	Expected Result
Order 2 Dosa; stock starts at 10	Accepted; bill Rs 100; Dosa stock becomes 8
Order 40 Tea; stock starts at 30	Rejected; stock unchanged
Quantity 0	Rejected as invalid quantity

9. Output Screenshot / Sample Execution

Sample console output for the example input. Run the program locally and capture a fresh screenshot if your instructor requires direct execution screenshots.

Sample Python console output

```
Number of order lines: 3
Item name (Idli/Dosa/Tea): Dosa
Quantity: 2
Order accepted. Bill: Rs 100
Item name (Idli/Dosa/Tea): Tea
Quantity: 3
Order accepted. Bill: Rs 45
Item name (Idli/Dosa/Tea): Idli
Quantity: 25
Order rejected: insufficient stock.

--- Daily Summary ---
Total sales: Rs 145
Remaining stock:
Idli: 20
Dosa: 8
Tea: 27
```

10. Conclusion

The canteen simulation connects billing with stock control. It uses lists, loops, conditions, and index-based updates to accept valid orders and preserve stock consistency.

FINAL OVERALL CONCLUSION

The ten selected case studies demonstrate how Python concepts can be applied to practical tasks. The solutions use variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, nested lists, tuples, and tuple operations where appropriate. Each study includes the mandatory documentation requested in the college circular.

Submission checklist

☐ Fill in student name, roll number, branch, and section on the cover.

☐ Run each program and verify output against the test cases.

☐ Capture genuine screenshots of your program execution.

☐ Create a GitHub repository named Python-Activity-Based-Learning.

☐ Upload the programs, documentation, screenshots, README.md, and consolidated PDF.

☐ Submit the repository URL to your faculty member / Lab Mentor on the first working day after the Dussehra holidays.

Student declaration: I confirm that I have reviewed, tested, and understood the programs included in this report and can explain them independently.

Student Signature: _____ Date: _____

CASE STUDY 12 — Attendance Recovery Computation

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Enter attended classes, total classes and target %

Are inputs valid?

If no: show “Invalid input” and stop

Is total classes zero?

STEP GROUP 2

If yes: show “Percentage undefined” and stop

Calculate current attendance %

Add future classes one by one until target % is reached

Display current %, extra classes needed and projected %

END

Read the left column from top to bottom, then continue at the top of the right column.

CASE STUDY 13 — ATM Transaction Simulation

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Enter opening balance and number of transactions

For each transaction, enter type (D/W) and amount

Is type valid and amount positive?

If no: reject transaction and count it

Deposit?

STEP GROUP 2

If yes: add amount to balance

For withdrawal: within limit and enough balance?

If no: reject; otherwise subtract amount

Update transaction counters; repeat for remaining transactions

Display final balance and transaction summary

END

Read the left column from top to bottom, then continue at the top of the right column.

CASE STUDY 17 — Student Performance Trend Analysis

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Prepare subjects and marks for each exam

Are all marks between 0 and 100?

If no: display invalid marks and stop

Calculate each exam total and average

STEP GROUP 2

Compare averages of consecutive exams

Calculate each subject’s average across exams

Display trend, strongest subject and weakest subject

END

Read the left column from top to bottom, then continue at the top of the right column.

CASE STUDY 27 — Hollow Diamond Pattern Generator

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Enter pattern width

Is width odd and between 3 and 19?

If no: display an error and stop

Find the middle row

For each row, calculate leading spaces and inner spaces

STEP GROUP 2

Print one star at the tip, otherwise print outline stars

Are more rows remaining?

If yes: calculate and print the next row

Finish the hollow diamond

END

Read the left column from top to bottom, then continue at the top of the right column.

CASE STUDY 29 — Password Policy Validation

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Enter password

Check minimum length (8 characters)

Check uppercase, lowercase, digit and special character

Check for 3 identical consecutive characters

STEP GROUP 2

Check whether “password” appears in any letter case

Did any rule fail?

If yes: list every failed rule

If no: display “All rules passed”

END

Read the left column from top to bottom, then continue at the top of the right column.

CASE STUDY 35 — Text Compression Using Character Runs

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Enter text

Is the text empty?

If yes: display empty result and stop

Scan text and count consecutive repeated characters

STEP GROUP 2

Build encoded text using each character and its count

Is encoded text shorter than original?

If yes: select encoded text; otherwise select original

Display encoded text, lengths and selected representation

END

Read the left column from top to bottom, then continue at the top of the right column.

CASE STUDY 42 — Student Marks Matrix Analysis

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Prepare student names, subjects and marks matrix

Are all marks between 0 and 100?

If no: display invalid marks and stop

For each student, calculate total and average

STEP GROUP 2

For each subject, calculate column average

Find student with highest total

Flag subjects with average below 50

Display student results, top student and flagged subjects

END

Read the left column from top to bottom, then continue at the top of the right column.

CASE STUDY 48 — Employee Shift Conflict Detection

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Prepare shift tuples: employee, start time, end time

Are all times valid (0 ≤ start < end ≤ 24)?

If no: report invalid shift data

Compare each pair of different shifts

Same employee and time intervals overlap?

STEP GROUP 2

If yes: report a shift conflict

Continue until all pairs have been checked

Were any conflicts found?

Display conflicts, or “All shifts are valid”

END

Read the left column from top to bottom, then continue at the top of the right column.

CASE STUDY 49 — Product Catalogue Audit

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Prepare product tuples: ID, price and quantity

Set total catalogue value to zero

Is product data valid?

If no: flag the invalid product record

If valid: calculate price × quantity and add to total

STEP GROUP 2

Is stock quantity below 5?

If yes: add product to replenishment list

Repeat for every product

Display product values, invalid records, reorder list and total

END

Read the left column from top to bottom, then continue at the top of the right column.

CASE STUDY 57 — Smart Canteen Order and Stock Management

CLEAR FLOWCHART • Follow the arrows from START to END

STEP GROUP 1

START

Set menu, prices and available stock

Enter number of orders

For each order, enter item and quantity

Does item exist and is quantity positive?

If no: reject the order

STEP GROUP 2

Is enough stock available?

If no: reject; if yes: calculate bill and reduce stock

Add accepted bill to daily sales; continue orders

Display daily sales and remaining stock

END

Read the left column from top to bottom, then continue at the top of the right column.